

FedGraft: Memory-Aware Heterogeneous Federated Learning via Model Grafting

Ruixuan Liu , Ming Hu , *Member, IEEE*, Zeke Xia, Xiaofei Xie , *Member, IEEE*, Jun Xia , *Member, IEEE*, Pengyu Zhang, Yihao Huang , *Member, IEEE*, and Mingsong Chen , *Senior Member, IEEE*

I. INTRODUCTION

Abstract—Although Federated Learning (FL) is good at collaborative learning among devices without compromising their data privacy, it suffers from the problem of large-scale deployment in Mobile Edge Computing (MEC) applications. This is mainly because the varying memory sizes of edge devices inevitably result in limited sizes of their hosting models. According to the Cannikin Law, when dealing with heterogeneous devices with different memory sizes, the learning capability of existing homogeneous FL schemes is greatly restricted by the weakest device. Worse still, although existing heterogeneous FL methods enable a MEC application to involve numerous devices equipped with heterogeneous models, their knowledge aggregation processes require either extra training data or architecture similarity of models. To address the above issues, this paper presents a novel FL method named FedGraft that enables effective knowledge sharing among heterogeneous device models of different sizes without imposing unrealistic assumptions. In FedGraft, all the device models are grafted to a common rootstock based on our proposed model partitioning and grafting mechanism, facilitating knowledge sharing among heterogeneous models on top of a tree-like global model. Meanwhile, using our proposed device selection strategy, the reassembled submodels extracted from the global model can be reasonably dispatched to corresponding devices with sufficient memory, thus enhancing the overall FL performance. Comprehensive experimental results show that, compared with state-of-the-art heterogeneous FL methods, FedGraft can improve inference accuracy by up to 17% in various memory-constrained scenarios.

Index Terms—Federated learning (FL), mobile edge computing (MEC), knowledge aggregation, memory constraint, model grafting.

Received 9 June 2024; revised 4 July 2025; accepted 16 July 2025. Date of publication 22 July 2025; date of current version 5 November 2025. This work was supported in part by the Natural Science Foundation of China under Grant 62272170, in part by the “Digital Silk Road” Shanghai International Joint Lab of Trustworthy Intelligent Software under Grant 22510750100, in part by Shanghai Trusted Industry Internet Software Collaborative Innovation Center, in part by National Research Foundation, Singapore, and in part by the Cyber Security Agency of Singapore under its National Cybersecurity R&D Programme and CyberSG R&D Cyber Research Programme Office. Recommended for acceptance by N. Shlezinger. (Corresponding authors: Ming Hu; Mingsong Chen.)

Ruixuan Liu, Zeke Xia, Pengyu Zhang, and Mingsong Chen are with the MoE Engineering Research Center of Hardware/Software Co-design Technology and Application, East China Normal University, Shanghai 200062, China (e-mail: mschen@sei.ecnu.edu.cn).

Ming Hu and Xiaofei Xie are with Singapore Management University, Singapore 188065 (e-mail: hu.ming.work@gmail.com).

Jun Xia is with the University of Notre Dame, Notre Dame, IN 46556 USA. Yihao Huang is with the National University of Singapore, Singapore 119077.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TMC.2025.3591537>, provided by the authors.

Digital Object Identifier 10.1109/TMC.2025.3591537

ALONG with the increasing popularity of Artificial Intelligence (AI) and Internet of Things (IoT) technologies, massive amounts of data are generated from mobile edge devices (e.g., smartphones, wearables, and sensors). To avoid uploading large amounts of private data directly to cloud servers for centralized training, Mobile Edge Computing (MEC) is proposed to migrate model training from the cloud to the edge. However, traditional machine learning methods make it difficult to meet the requirements of data privacy and security [1]. Driven by such realistic issues, Federated Learning [2], [3], [4], [5], [6] is becoming a new collaborative learning paradigm in designing various MEC applications (e.g., autonomous driving [7], [8], real-time computing [9], [10], [11], multi-agent control [12]). By adopting the client-server architecture, edge devices (i.e., clients) periodically upload the gradient information of their deep models to a specific cloud server relying on a global model for knowledge aggregation, and update their local knowledge based on the newly aggregated global model. Although FL facilitates knowledge sharing among devices without compromising their data privacy [13], it still suffers greatly from the problem of large-scale deployment. This is mainly because existing mainstream FL methods assume that all edge devices have sufficient memory size to host the deep models. However, this is not true in practice. In addition, an MEC application typically involves numerous deep models with heterogeneous architectures depending on their specific application purposes or resource constraints.

When taking into account the hardware heterogeneity of edge devices [14], [15], the overall inference performance of homogeneous FL will be directly affected by such factors. Specifically, the varying memory sizes of edge devices correspondingly limit the sizes of their hosting models [16], [17], [18], [19]. As a result, according to the Cannikin law, the inference performance of homogeneous FL methods will inevitably be affected. On the one hand, before the FL training, the global model selection is determined by devices with the least memory size. On the other hand, during the knowledge aggregation process, the overall learning performance of existing FL methods is often restricted by the smallest (weakest) device models. Some existing heterogeneous FL methods enable an MEC application to involve numerous edge devices equipped with heterogeneous models. They can be mainly classified into two categories, i.e., Knowledge Distillation (KD)-based [20], [21] and model pruning-based methods [22], [23], [24], [25]. However, most of

these approaches are built on unrealistic assumptions, i.e., KD-based methods require extra public training data, and pruning-based methods only work when there are architectural similarities between device models, thus severely limiting the pervasive usage of FL in real-world MEC applications. Therefore, how to enable unconditional and high-performance knowledge sharing among memory-constrained heterogeneous edge devices is becoming an urgent issue in the design of large-scale MEC systems.

Since knowledge transfer [26], [27], [28] allows knowledge sharing from deep models of a given domain to those of other domains, it has been investigated in the design of MEC applications (including FL-based applications [21], [29]) involving various heterogeneous models. To facilitate the knowledge transfer under both the hardware resource and performance constraints, a novel knowledge transfer task named Deep Model Reassembly (DeRy) [30] has been developed for general-purpose model reuse. In DeRy, a given set of heterogeneous models with diverse architectures can be partitioned into functionally equivalent and interchangeable building blocks (e.g., feature extraction and classification blocks) with specific functions, and such blocks can be selectively reassembled to derive customized models with high classification accuracy. Inspired by the model reassembly concept of DeRy, this paper proposes a novel FL method named FedGraft that enables unconditional collaborative training among heterogeneous models under the constraint of device memory sizes. In our approach, we divide each heterogeneous candidate model into two functional blocks, i.e., the *feature-extraction block* and the *device-adaptation block*, denoting the early and late parts of the model network, respectively. According to [31], the early layers of a network tend to capture low-level general features, which means that deep models with similar early parts are more likely to extract common features of the input samples that can be shared by heterogeneous models for initial knowledge extraction. To enable knowledge sharing among such heterogeneous models, our FedGraft method adopts a tree-like global model, where the tree's rootstock denotes the common feature-extraction block shared by all the grafted device models, and all the remaining device-adaptation blocks are grafted to the rootstock. Based on our proposed knowledge aggregation mechanism and device selection strategy, the overall inference potential of FedGraft can be fully explored under the constraint of device memory sizes. This paper makes the following three major contributions:

- We introduce a novel model partitioning and grafting mechanism to form a tree-like global model, on which our proposed block-wise aggregation scheme enables unconditional collaborative learning among heterogeneous models with different architectures.
- We developed an effective device selection strategy to encourage all submodels derived from the global model to participate in local training, thus sufficiently exploring the potential to improve overall FL performance.
- We prove the convergence of our FedGraft approach and conduct extensive experiments to evaluate its effectiveness based on various heterogeneous devices and models.

Experimental results on simulation-based and real test-bed-based platforms demonstrate the superiority, adaptivity,

scalability, and applicability of FedGraft within various Independent and Identically Distributed (IID) and non-IID scenarios.

The remainder of this paper is organized as follows. After introducing related work in Section II, Section III presents preliminary knowledge of FL and model partitioning, Section IV presents the details of the implementation of FedGraft, and performs the convergence analysis. Section V evaluates the effectiveness of FedGraft against previous methods on three well-known datasets. Finally, Section VI concludes the paper.

II. RELATED WORK

Although FL is becoming increasingly popular in designing MEC applications, it still suffers from low classification performance when faced with various heterogeneity issues (e.g., heterogeneous models and devices). Worse still, conventional homogeneous FL methods are hard to adapt to large-scale applications since they assume that all devices have sufficient memory. To address the issue of heterogeneity, various heterogeneous FL methods have been developed, which can be mainly classified into two categories, i.e., Knowledge Distillation (KD)-based and model pruning-based methods.

Based on knowledge distillation [32], KD-based FL methods [20], [21], [33], [34] enable knowledge sharing among heterogeneous models. For example, FedMD [20] uses the average of outputs of heterogeneous local models to guide the model training through distillation with a global proxy dataset shared among all the clients. FedDF [21] leverages the ensemble of output logits from heterogeneous local models to distill the aggregated global model with a proxy dataset on the server. Although these KD-based methods are promising for heterogeneous FL, they require a public proxy dataset during FL training, which is hard to obtain in practice and may inevitably compromise the data privacy of real applications.

As an alternative, model pruning-based methods [22], [23], [35], [36] solve the problem of model heterogeneity by trimming the global model into heterogeneous sub-models. For example, HeteroFL [22] and FedMP [37] prune the global model into sub-models of different widths to fit clients with different memory sizes and aggregate overlapping sub-model parameters to share knowledge. Unlike these two methods, DepthFL [23] prunes the global model into sub-models of different depths. As a two-dimensional pruning approach, ScaleFL [36] utilizes self-distillation to transfer knowledge among submodels. Although such pruning-based approaches can effectively reduce the sizes of local models trained on clients, their inference performance is greatly limited, since the pruning schemes can easily destroy model structures and result in the parameter mismatch problem during model aggregation. To mitigate such impacts of model pruning, various remedies were developed. For example, instead of pruning models, InclusiveFL [38] and FlexiFed [29] improve overall inference accuracy by only aggregating the common parts of heterogeneous models. However, these methods are also based on the assumption of model similarities.

Inspired by the model reassembly concept of DeRy [30], this paper explores an FL method that enables unconditional and high-performance knowledge sharing among memory-constrained heterogeneous devices. Note that DeRy is designed

primarily for traditional deep learning tasks, assuming that all training data can be accessed to determine the similarity information of model blocks. However, this assumption is not true for FL scenarios, especially in non-IID scenarios. To the best of our knowledge, our FedGraft approach is the first attempt to leverage the model partitioning and grafting mechanism to achieve this goal. Compared with state-of-the-art heterogeneous FL methods, FedGraft can not only conduct heterogeneous FL training without requiring any extra training datasets, but also maximally maintain the structural information of original heterogeneous models based on a tree-like global model.

III. PRELIMINARIES

Federated Learning: Typically, FL adopts a client-server architecture, where the cloud server hosts a global model with the same architecture as the models of clients for knowledge aggregation. By iteratively averaging the gradients of models uploaded by clients, the updated global model will be dispatched to clients to enable knowledge sharing among their models. Assume that there are a total of N clients involved in FL, among which K ($K \leq N$) clients participate in knowledge aggregation in each FL communication round. Let each client k have its private local dataset D_k following the distribution $P_k(x, y)$, where x and y denote the input features and the corresponding class labels, respectively. To perform local training, each client uses the objective function:

$$f_k(w_k) = \frac{1}{|D_k|} \sum_{i=1}^{|D_k|} \ell(w_k; x_i, y_i), \quad (1)$$

where $|D_k|$ represents the number of samples in D_k , w_k is the parameters of a local model on the k^{th} client, $(x_i, y_i) \in D_k$, and ℓ denotes the loss function (e.g., the cross-entropy loss) of an adopted supervised learning task. The goal of FL is to enable knowledge sharing among clients via a global model hosted by the server, whose objective function is as follows:

$$\arg \min_{\bar{w}} F(\bar{w}) = \sum_{k=1}^N \frac{|D_k|}{|D|} f_k(\bar{w}) \quad (2)$$

where $|D|$ is number of all samples on N clients, and $\bar{w}$ is the parameters of the global model.

Model Partitioning: According to DeRy [30], a model can be divided into multiple building blocks based on the feature similarity using the following input-output similarity function:

$$S(B, B') = s(B(x), B'(x')) + s(x, x') \quad (3)$$

where $B(x)$ denotes the output feature of x via a block B , and $s(x, y)$ measures the similarity between x and y based on the Centered Kernel Alignment (CKA) [39]. For a given model zoo Z , our approach assumes that each model $m_i \in Z$ can be divided into two disjoint parts B_i^0 and B_i^1 using the following K-means-style optimization function:

$$\arg \max_{f_i \in \{1, \dots, |L_i|\}} \sum_{i=1}^{|Z|} \left[S(B_i^{0, f_i}, B_{an}^{0, f_i}) + S(B_i^{1, f_i}, B_{an}^{1, f_i}) \right], \quad (4)$$

where $|Z|$ indicates the number of models, $|L_i|$ indicates the number of total layers of the model m_i , and f_i denotes the indices

of model layers of the i^{th} model. Let B_{an}^{0, f_i} and B_{an}^{1, f_i} be the anchor node blocks for the sets of former divided parts $B^{0, f}$ and latter divided parts $B^{1, f}$, respectively, indicating the block that has the maximum summed similarities with all the other blocks in the same set. We use B_i^{0, f_i} and B_i^{1, f_i} to represent the first divided part of m_i containing the 1^{th} - f_i^{th} layers and the second divided part containing the remaining layers, respectively. By Equation 4, we can get an optimal model partitioning for the models in the zoo.

IV. OUR FEDGRAFT APPROACH

A. Overview

Fig. 1 outlines our FedGraft approach, which consists of two major parts, i.e., the cloud server and all the selected (activated) edge devices within one FL communication round. To adapt to various heterogeneous devices with varying memory sizes, FedGraft keeps a *Model and Client Selector* (MCS) in the cloud server, which maintains the memory usage information of all participating edge devices. Such information can be used to select candidate models for heterogeneous model derivation and guide the assignment of reassembled submodels extracted from the global model to activated devices.

As shown in the figure, the FedGraft workflow involves two stages: i) the initial model generation stage, and ii) the FL training stage. In the initial model generation stage, the server first collects a set of candidate models appropriate for all involved devices, which forms a model zoo according to the device memory information maintained by the MCS. Then, the server partitions the models in the model zoo based on (4), where each candidate model is divided into two blocks, i.e., the feature-extraction block and the device-adaptation block. After that, the server chooses one device-adaptation block as the rootstock for grafting all the device-adaption blocks based on our designed model adapters, thus constricting a global model with a tree-like structure. For the FL training stage, the server decomposes the tree-like global model and extracts corresponding heterogeneous submodels with different sizes. Note that all the submodels share the same rootstock (i.e., the feature-extraction block) for local training. Based on the device memory information recorded in the MCS, all these submodels are dispatched to selected devices for local training in the same way as FedAvg, where the dispatched models are guaranteed to be no larger than the available memory of their designated devices. At the end of each FL communication round, all the trained local models will be sent to the server for knowledge aggregation based on our proposed model aggregation scheme. The whole process of FL training terminates until the convergence of the tree-like global model. The following subsections will detail the key components and steps of our FedGraft method.

B. Initial Model Generation

When dealing with a MEC application involving numerous memory resource-constrained devices, FedGraft aims to generate a set of heterogeneous models to enable collaborative training among such heterogeneous devices without imposing unrealistic assumptions on extra training data or structural similarities

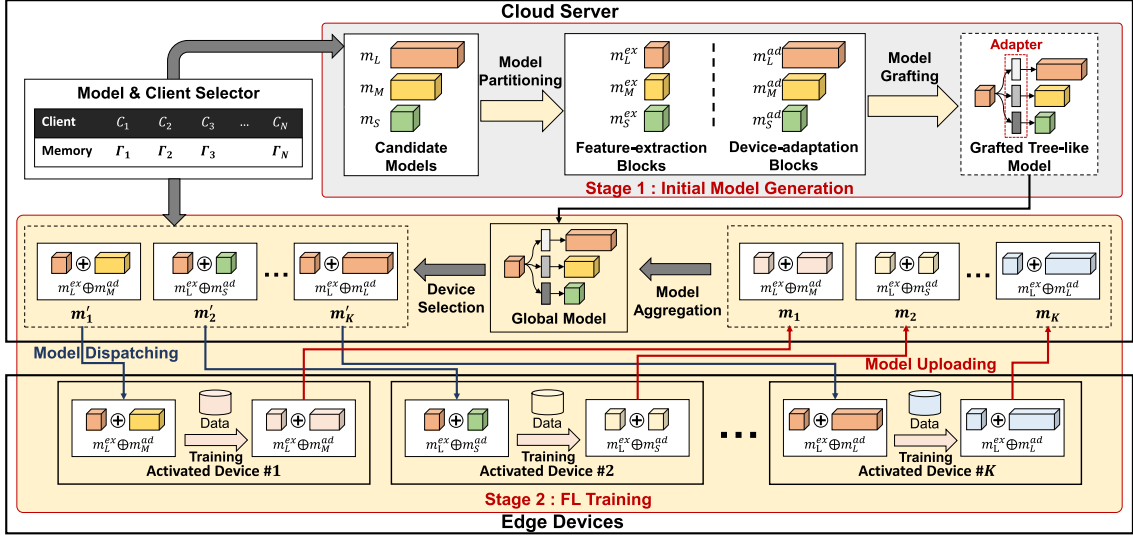


Fig. 1. The framework and workflow of FedGraft.

between models. As shown in Fig. 1, the MCS of the server maintains a table to record the memory size information of each involved device. To facilitate the collaborative training, we classified all devices into distinct levels (e.g., weak/medium/strong devices) according to their available memory capacities. Meanwhile, based on the classified device levels, MCS will determine appropriate candidate models (e.g., small/medium/large models) for edge devices with different levels. As an example shown in Fig. 1, there are three heterogeneous candidate models in the model zoo Z , i.e., m_L , m_M , and m_S , representing the large, medium, and small models, respectively. According to (4), the server partitions each candidate model m_i into two parts, i.e., the feature-extraction block m_i^{ex} and device-adaptation block m_i^{ad} , via the function $Partitioning()$. Then, the server selects one m_i^{ex} as the rootstock and grafts all the m_i^{ad} blocks on it using the function $Grafting()$. This way, the server initializes a tree-like global model with branches of different sizes.

1) *Model Partitioning*: For conventional heterogeneous FL, it is difficult to conduct collaborative training directly on heterogeneous models without resorting to extra training data or model similarities, since there is a lack of entries or mediums to enable knowledge sharing among models. Rather than conducting FL training on complete models, our model partitioning method strives to explore the potential of knowledge sharing between fine-grained model modules. Our approach is built on top of the following two facts: i) a given set of heterogeneous models with diverse architectures can be partitioned into functionally equivalent and interchangeable building blocks [30], and ii) the early layers of models are used to capture common features of the input samples [31]. The basic idea of our model partitioning method is to figure out a common rootstock based on the early layers of some model for all heterogeneous models, which enables the grafting of all the remaining parts of models onto the rootstock. During local training, FedGraft can easily construct a new local model based on the rootstock and a specific branch

(i.e., a device-adaption block), satisfying the memory size requirement of a selected target device. Since the initial knowledge extracted by the same rootstock is regularized, FedGraft enables knowledge sharing among all the submodels.

When performing model partitioning, we adopt the CKA [39] as the similarity metric and divide the candidate models in a given zoo based on (4) as follows:

$$\begin{aligned}
 Partitioning(Z) &= \{ \langle m_i^{ex}, m_i^{ad} \rangle \}_{i=1}^{|Z|} \\
 &= \{ \langle m_i^{ex, f_i}, m_i^{ad, f_i} \rangle \}_{i=1}^{|Z|} \\
 \text{s.t. } \arg \max_{f_i \in \{1, \dots, |L_i|\}} &\sum_{i=1}^{|Z|} [CKA(m_i^{ex, f_i}, m_{an}^{ex, f_i}) \\
 &+ CKA(m_i^{ad, f_i}, m_{an}^{ad, f_i})]. \quad (5)
 \end{aligned}$$

After the partitioning, we can get two sets of blocks with functional similarities, where E is the set of feature-extraction blocks, and A is the set of device-adaptation blocks. Note that, according to the observation in DeRy [30], the blocks in A generally have much more layers and parameters than the ones in E , i.e., the rootstock of a tree-like global model is much smaller than its branches. This property is benign for heterogeneous FL, since all devices can accommodate any selected rootstock.

2) *Model Grafting*: Our model grafting method aims to re-assemble the functional blocks in A and E to enable knowledge sharing among heterogeneous devices with different memory sizes. To achieve this goal, FedGraft adopts a tree-like global model M_i , where the rootstock m_i^{ex} comes from E determined by the function $SelectRootstock()$ and uses all the device-adaptation blocks in A as branches grafted onto the rootstock. In our tree-like global model, the knowledge extracted by the common rootstock can be forwarded to the grafted branches, enabling knowledge-sharing among grafted models without imposing assumptions on extra training data or model similarities.

During model grafting, since the selected rootstock (i.e., the feature-extraction block) and a grafted device-adaptation block may come from different candidate models, their feature dimensions may mismatch. To address this issue, for each pair of the common rootstock and a device-adaptation block, we propose using a lightweight neural network with only two convolutional layers as an adapter to align the input/output dimensions of the two blocks. As an example shown in Fig. 1, by using three adapters, the server initializes a tree-like global model based on one feature-extraction block (i.e., m_L^{ex}) and three device-adaptation blocks (i.e., m_S^{ad} , m_M^{ad} , m_L^{ad}) based on our *Grafting()* function defined as follows:

$$\begin{aligned} Grafting(E[i], A) &= E[i] \oplus A = m_i^{ex} \oplus \begin{pmatrix} m_S^{ad} \\ m_M^{ad} \\ m_L^{ad} \end{pmatrix} \\ &= \left\langle (m_i^{ex} \circ \alpha^0) \circ \begin{pmatrix} \alpha_S^1 \circ m_S^{ad} \\ \alpha_M^1 \circ m_M^{ad} \\ \alpha_L^1 \circ m_L^{ad} \end{pmatrix} \right\rangle, \quad (6) \end{aligned}$$

where m_i^{ex} and $m_S^{ad}, m_M^{ad}, m_L^{ad}$ represent the i^{th} feature-extraction block in E , and the device-adaptation blocks of small/medium/large devices, respectively. Here, we use α^0 and α^1 to denote the first and second convolutional layers of an adapter α , respectively. For example, we use the pair of α^0 and α_L^1 to construct an adapter between the rootstock m_i^{ex} and m_L^{ad} , which aligns the input/output dimensions of m_i^{ex} and m_L^{ad} . Note that all adapters share the same first convolutional layer and are automatically generated.

C. Device Selection and Model Dispatching

Let $M_i = Grafting(E[i], A)$ be the tree-like global model for FL training, where we can derive a set of reassembled submodels for local training. In the device selection step, we divide the global model into the rootstock, adapters, and device-adaptation blocks. Assuming that $Z = \{S, M, L\}$ and the global model only has three device-adaptation blocks (i.e., $m_S^{ad}, m_M^{ad}, m_L^{ad}$), we can derive three heterogeneous submodels (i.e., $\tilde{m}_{i,j}$ indicating small, medium, and large models with a rootstock from m_i^{ex} , respectively, where $j \in Z$) using the following function:

$$\begin{aligned} Extracting(M_i) &= \bigcup_{j \in Z} \tilde{m}_{i,j} = \bigcup_{j \in Z} (E[i] \oplus m_j^{ad}) \\ &= \bigcup_{j \in Z} (m_i^{ex} \oplus m_j^{ad}) \\ &= \bigcup_{j \in Z} (m_i^{ex} \circ \alpha^0) \circ (\alpha_j^1 \circ m_j^{ad}). \quad (7) \end{aligned}$$

Under the guidance of the MCS, we dispatch the reassembled submodels to selected devices in each FL communication round. In the MCS, FedGraft maintains a table that monitors the available memory Γ_k of each device k and determines which submodels will be dispatched to the selected devices via the function *Selecting()*. In each FL communication round, FedGraft uses a model list G to keep such model deployment information, where $G[k]$ denotes a submodel $\tilde{m}^k$ dispatched to the k^{th} selected device. Let $|\tilde{m}_{i,j}|$ be the required memory for the

deployment of $\tilde{m}_{i,j}$. When dispatching a submodel $\tilde{m}_{i,j}$ to some device k , the condition $|\tilde{m}_{i,j}| \leq \Gamma_k$ should hold. In other words, each submodel can only be dispatched to a device with sufficient memory to accommodate it. For example, assume that there are three kinds of devices (i.e., strong/medium/weak devices) that just fit large/medium/small models, respectively. In this case, the small submodel can be deployed on all strong/medium/weak devices. Meanwhile, the medium submodel can be deployed on strong/medium devices, while the large submodel can only be deployed on strong devices. To achieve sufficient knowledge sharing among all dispatched models, in each FL communication round, FedGraft dispatches selected models according to the ratio of heterogeneous devices Ra . For example, assume that the total number of devices is n , the percentage of selected devices in each round is p , and the ratio of weak/medium/strong devices Ra is $x:y:z$. In each round, FedGraft will consider $n \times p$ devices, and for each model, we will select a suitable device that can accommodate the model. In other words, we will select at least $\frac{n \times p \times z}{x+y+z}$ devices from all strong devices to hold the large submodel, $\frac{n \times p \times y}{x+y+z}$ devices from the strong/medium devices to hold the medium submodel and $\frac{n \times p \times x}{x+y+z}$ devices from all strong/medium/weak devices to hold the small submodel.

D. Block-Wise Aggregation

In one FL communication round, after local training based on the model dispatching scheme represented by G , we need to upload the parameters of locally trained models back to the server for knowledge aggregation. Since all reassembled submodels are heterogeneous, conducting the conventional FedAvg-like aggregation (see (2)) on them is impossible. Due to the submodel structure shown in (7), all uploaded models have the same feature-extraction part. According to the work in Dery [30], this enables us to perform the model aggregation based on building blocks. In FedGraft, we proposed to use block-wise aggregation to share knowledge among heterogeneous submodels, where we separately calculate the parameters of the feature-extraction block w_i^{ex} and different device-adaptation blocks w_j^{ad} to figure out the overall parameters of the global model M_i .

Assume that each FL round involves K selected devices. Since all the uploaded models share the same feature-extraction block, similar to FedAvg-like aggregation, the parameters w^{ex} of the rootstock and the first layer of its adapter (i.e., α^0) of the updated tree-like global model can be calculated as:

$$AggregatingEx(G, K) = \frac{1}{K} \sum_{k=1}^K w_k^{ex}, \quad (8)$$

where w_k^{ex} is the parameters of the feature-extraction block from the trained submodel $\tilde{m}^k$ dispatched to the k^{th} selected device in that round. Unlike the above calculation for the rootstock, FedGraft only aggregates device-adaptation blocks with the same structure and their specific adapter. Assume that C_j is the set of selected devices whose local models have the same structure as the one of $\tilde{m}_{i,j}$. Let $w_{i,j}^{ad}$ be the parameters of both the device-adaptation block (i.e., m_j^{ad}) and the second layer of its adapter (i.e., α_j^1) in the updated tree-like global model. The

Algorithm 1: Implementation of FedGraft.

Input: i) T , the total number of training rounds; ii) C , the set of edge devices; iii) Γ , client memory thresholds; iv) K , the number of selected devices for local training in each round; v) $Z = \{m_1, m_2, \dots, m_{|Z|}\}$, the candidate model zoo; vi) Ra , the ratio of heterogeneous devices.

Output: M_i , the i^{th} tree-like global model.

```

1: /* Stage1: Initial Model Generation */
2:  $\{ \langle m_i^{ex}, m_i^{ad} \rangle \}_{i=1}^{|Z|} \leftarrow \text{Partitioning}(Z)$ 
3:  $E \leftarrow \{m_1^{ex}, m_2^{ex}, \dots, m_{|Z|}^{ex}\}$ 
4:  $A \leftarrow \{m_1^{ad}, m_2^{ad}, \dots, m_{|Z|}^{ad}\}$ 
5:  $i \leftarrow \text{SelectRootstock}(E)$ 
6:  $M_i \leftarrow \text{Grafting}(E[i], A)$ 
7: /* Stage2: FL Training */
8: for each round  $t \leftarrow 1, \dots, T$  do
9:    $R = \{\tilde{m}_{i,1}, \tilde{m}_{i,2}, \dots, \tilde{m}_{i,|Z|}\} \leftarrow \text{Extracting}(M_i)$ 
10:   $G = [\tilde{m}^1, \tilde{m}^2, \dots, \tilde{m}^K] \leftarrow \text{Selecting}(C, \Gamma, K, R, Ra)$ 
11:  /* Client Side */
12:  for each client  $k \leftarrow 1, \dots, K$  in parallel do
13:     $\tilde{m}^k \leftarrow \text{LocalTraining}(D_k, \tilde{m}^k)$ 
14:     $G[k] \leftarrow \tilde{m}^k$ 
15:  end for
16:  /* Cloud Server Side */
17:   $m_i^{ex} \leftarrow \text{AggregatingEx}(G, K)$ 
18:  for each  $j \leftarrow 1, \dots, |Z|$  do
19:     $m_j^{ad} \leftarrow \text{AggregatingAd}(G, i, j)$ 
20:     $A[j] \leftarrow m_j^{ad}$ 
21:  end for
22:   $M_i \leftarrow \text{Merging}(m_i^{ex}, A)$ 
23: end for
24: return  $M_i$ 

```

parameters $w_{i,j}^{ad}$ of the j^{th} branch of the tree-like global model can be calculated using the following function:

$$\text{AggregatingAd}(G, i, j) = \frac{1}{|C_j|} \sum_{k \in C_j}^{C_j} w_{i,j,k}^{ad}, \quad (9)$$

where $w_{i,j,k}^{ad}$ is the parameters of the device-adaptation block from the trained submodel $\tilde{m}_{i,j}^k$ in the k^{th} device. After the above block-wise aggregation methods based on the functions $\text{AggregatingEx}()$ and $\text{AggregatingAd}()$, we can obtain an updated feature-extraction block m_i^{ex} , and all the updated branches (i.e., all device-adaptation blocks in A coupled with their adapters). Then, by using the function $\text{Merging}()$ to concatenate all these blocks, we can form a newly updated tree-like global model. Based on the block-wise aggregation results, FedGraft can enhance the knowledge of the tree-like global model M_i , which in turn improves the knowledge transfer among different submodels. Note that our approach does not impose any requirement on additional training data.

E. Implementation of FedGraft

Algorithm 1 details the implementation of our FedGraft approach, which consists of two stages, i.e., the initial model generation stage and the FL training stage. In the first stage (Lines 2-6), Lines 2-4 partition the candidate models in the zoo based on (5) and form two sets E and A to keep all the feature-extraction blocks and device-adaption blocks, respectively. Line 5 determines a suitable rootstock from E for the model grafting based on the optimal partition. Then, Line 6 constructs a tree-like global model using the rootstock and all the device-adaption blocks in A , which enables collaborative FL learning among heterogeneous devices. Lines 8-23 present the FL training process involving the iterative interaction between selected clients and the server. At the beginning of each FL communication round, Line 9 first extracts the local models based on M_i . Then, based on the given ratio of heterogeneous devices, Line 10 selects suitable devices that can accommodate them and dispatches the models to them. For the client side, Lines 12-15 conduct local training for each selected device. Based on the local data D_k of the k^{th} device, the parameters of its local model $\tilde{m}^k$ are updated in Line 14. For the cloud server side, Lines 17-22 perform the knowledge aggregation using our proposed block-wise aggregation mechanism. In the same way as FedAvg, Line 17 aggregates all the rootstocks of the uploaded local models. Lines 18-21 separately aggregate the device-adaptation blocks (together with their specific adapters) with the same structures. Then, step 22 merges all the updated blocks and forms a new tree-like global model. Finally, Line 24 reports the collaboratively trained tree-like global model M_i as the output.

F. Convergence Analysis

Inspired by the work of FedAvg [40], our convergence analysis for FedGraft is as follows.

1) *Notations:* Unlike traditional FL methods, FedGraft adopts a tree-like supernet as the global model, from which reassembled submodels are derived and sent to selected devices for local training in each communication round. To facilitate the convergence analysis, we mimic the above process by dispatching the whole global model to each selected client and only updating its parameters corresponding to the ones of an assigned submodel to the client during the local training. We assume all clients in FedGraft adopt Stochastic Gradient Descent (SGD) as the optimizer, and there are K devices selected for knowledge aggregation in each FL communication round. Let w_t^k ($1 \leq k \leq K$) be the parameters of a tree-like client model hosted by the k^{th} selected device trained at the t^{th} global SGD step, where $w_t^k = w_t^{ex,k} \oplus \bigcup_{j \in \{S,M,L\}} w_{t,j}^{ad,k}$ represents the concatenation of the rootstock (i.e., the feature-extraction block $w_t^{ex,k}$) and the grafted device-adaptation blocks $w_{t,j}^{ad,k}$ ($j \in \{S, M, L\}$).

Let $v_{t+1}^k = v_{t+1}^{ex,k} \oplus \bigcup_{j \in \{S,M,L\}} v_{t+1,j}^{ad,k}$ be the immediate result of w_t^k after executing one SGD step, where $v_{t+1}^{ex,k}$ and $v_{t+1,j}^{ad,k}$ denote the immediate results of the feature-extraction block and

device-adaptation blocks, respectively. Assuming that each FL round involves E SGD steps, we have

$$w_{t+1}^k = \begin{cases} v_{t+1}^k, & \text{if } (t+1)\%E \neq 0 \\ \frac{1}{K} \sum_{k=1}^K v_{t+1}^k, & \text{if } (t+1)\%E = 0 \end{cases} \quad (10)$$

where the condition $(t+1)\%E = 0$ indicates the last SGD step within one FL round. The update of the global model in one SGD step involves two parts as follows.

- 1) After the feature-extraction block $v_t^{ex,k}$ performs one forward propagation with a data batch ξ_t^k , we can get an intermediate feature $y_c = v_t^{ex,k}(\xi_t^k)$.
- 2) For a specific selected grafted device-adaptation block $v_{t,j}^{ad,k}$ ($j \in \{S, M, L\}$), we continue the forward propagation using y_c and get $y = v_{t,j}^{ad,k}(y_c)$. Then, we update the device-adaptation block based on the backward propagation using the formula $v_{t+1,j}^{ad,k} = v_{t,j}^{ad,k} - \eta_t \nabla(\ell(y))$, where ℓ denotes the loss function of this branch. Note that all the other device-adaptation blocks are not updated at this SGD step. After updating the device-adaptation block, we update the feature-extraction block using the formula $v_{t+1}^{ex,k} = v_t^{ex,k} - \eta_t \nabla(\frac{\partial \ell(y)}{\partial y_c})$.

To facilitate the proof, we use $g_t^k = \nabla f_k(w_t^k; \xi_t^k) = \text{Concatenate}(\nabla(\frac{\partial \ell(y)}{\partial y_c}), \nabla(\ell(y)))$ to denote the model gradients of the k^{th} client after training a data batch ξ_t^k at the t^{th} SGD step, and update v_{t+1}^k with g_t^k by $v_{t+1}^k = w_t^k - \eta_t g_t^k$.

Let $\bar{v}_t$ and $\bar{w}_t$ be the aggregated versions of the parameters of all the participating models, i.e., v_t^k and w_t^k ($k \in \{1, 2, \dots, K\}$), respectively. Note that, in FedGraft, the server sends the whole tree-like global model to all K selected clients in each communication round. Similar to FedAvg, we can get $\bar{v}_t$ and $\bar{w}_t$ using the following formulas:

$$\bar{v}_t = \frac{1}{K} \sum_{k=1}^K v_t^k, \quad \bar{w}_t = \frac{1}{K} \sum_{k=1}^K w_t^k.$$

2) *Assumptions*: We analyze the convergence of FedGraft based on the following four assumptions about the loss functions of the selected client (i.e., $f_1, \dots, f_K$) at each SGD step. Similar to the definitions in [41], [42], Assumption 2 denotes the L -smooth assumption, Assumption 3 is the μ -convex assumption, and Assumption 4 indicates the variance/mean bound assumption for stochastic gradients. Here, w and w' are both model parameters, where w' is a successor of w after applying multiple SGD steps.

Assumption 1: In the t^{th} global SGD step of FedGraft, after receiving a tree-like global model w_t^k from the server, the k^{th} ($1 \leq k \leq K$) selected client conducts local training based on the j^{th} branch $w_{t,j}^{ad,k}$ (i.e., the device-adaptive block of an assigned submodel indicated by G), while the learning rates of other branches are set to 0.

Assumption 2: The loss f_k is L -smooth satisfying $f_k(w) \leq f_k(w') + (w - w')^T \nabla f_k(w') + \frac{L}{2} \|w - w'\|_2^2$, $k \in \{1, \dots, K\}$.

Assumption 3: The loss f_k is μ -convex satisfying $f_k(w) \geq f_k(w') + (w - w')^T \nabla f_k(w') + \frac{\mu}{2} \|w - w'\|_2^2$, where $k \in \{1, \dots, K\}$ and $\mu \geq 0$.

Assumption 4: The variance of stochastic gradients is upper bounded by σ^2 . The expectation of squared norm of stochastic gradients is upper bounded by G^2 , i.e., $\mathbb{E} \|\nabla f_k(w_t^k; \xi_t^k) - \nabla f_k(w_t^k)\|^2 \leq \sigma^2$, $\mathbb{E} \|\nabla f_k(w_t^k; \xi_t^k)\|^2 \leq G^2$ for all $k \in \{1, \dots, K\}$, where ξ_t^k is a data batch of the k^{th} client in the t^{th} SGD step.

3) *Proof of Convergence*: Assume that Assumptions 1–4 hold. We prove the convergence of FedGraft using Theorem 1 as follows. Please refer to the Appendix for the proof details, available online.

Theorem 1: Assume that FedGraft training requires r communication rounds, where each round consists of E SGD steps. Let $t = r \times E$ be the index of the current SGD step, w_t be the current output model, w^* be the parameters of an optimal model, and $\eta_t = \frac{2}{\mu(t+\lambda)}$ be the learning rate. We have:

$$\mathbb{E}[F(\bar{w}_t)] - f^* \leq \frac{L}{2\mu(t+\lambda)} \left[\frac{4B}{\mu} + \frac{\mu(\lambda+1)}{2} \Delta_1 \right],$$

where $B = 4L\Gamma + 4(E-1)^2G^2$ and $\Delta_1 = \|\bar{w}_1 - w^*\|^2$.

From Theorem 1, we can find that the difference between the current loss $F(\bar{w}_t)$ and the optimal loss f^* decreases and tends to 0 as t increases, indicating that the global model in FedGraft will eventually converge.

V. PERFORMANCE EVALUATION

To evaluate the performance of FedGraft, we conducted extensive experiments on various well-known datasets and models using two platforms: i) a simulation-based platform in the form of an Ubuntu workstation equipped with an Intel i9 CPU, 32 GB of memory, and an NVIDIA RTX 3090Ti GPU, and ii) a real test-bed platform that includes 8 Jetson Nano boards and 2 Jetson Xavier AGX boards as devices and an Ubuntu workstation with NVIDIA RTX 3080 GPU as the server. We implemented FedGraft on top of Pytorch (version 2.0.1), which aims to answer the following four Research Questions (RQs).

RQ1 (Superiority of FedGraft): What are the advantages of FedGraft compared with state-of-the-art FL methods?

RQ2 (Adaptability of FedGraft): What is the adaptivity of FedGraft to different settings (e.g., dataset and DNN settings)?

RQ3 (Scalability of FedGraft): What is the performance of FedGraft under large-scale deployment?

RQ4 (Applicability of FedGraft): What is the performance of FedGraft within real-world MEC scenarios?

A. Experimental Settings

In our experiments, we assumed that there are 100 devices involved in the FL by default, among which 10% of devices are selected for knowledge aggregation in each communication round. We set the batch size to 50 for the local training and conducted 5 epochs in each FL round. We adopted Stochastic Gradient Descent (SGD) [43] as the optimizer with a learning rate of 0.01, a learning rate decay of 0.998, a momentum of 0.5, and a weight decay of $1e^{-3}$ for all the investigated FL methods in our experiments.

TABLE I
STATISTICS OF DIFFERENT MODEL PARTITIONS

Model	CNN		MobileNetV2		ResNet18		VGG16	
Block	m^{ex}	m^{ad}	m^{ex}	m^{ad}	m^{ex}	m^{ad}	m^{ex}	m^{ad}
Layers	1-1	2-2	1-19	20-54	1-9	10-18	1-4	5-16
Params (M)	0.01	0.20	0.06	2.19	0.67	10.50	0.26	33.39
FLOPs (G)	0.05	0.07	0.60	1.78	14.45	13.45	4.85	11.82
	0.12		2.38		27.89		16.67	

1) *Dataset Settings*: We conducted experiments on three well-known datasets (i.e., CIFAR-10, CIFAR-100, and TinyImageNet) involving IID and non-IID scenarios. For each dataset, we adopted the Dirichlet distribution [44] denoted by $Dir(\beta)$ to control the heterogeneity of client data, where a smaller β indicates the higher heterogeneity of client data.

2) *Model Settings*: By default, our experiments were carried out based on three models, i.e., classical CNN (with two convolution layers), MobileNetV2 [45] and ResNet18 [46], representing small, medium and large candidate models, respectively. To evaluate the performance of FedGraft with different model architectures, we investigated another combination of models, including MobileNetV2, ResNet18, and VGG16 [47], in the order of the number of parameters.

Table I shows the partitioned blocks coupled with their statistics (i.e., the layers, number of parameters, and FLOPs of blocks) of the four investigated models with different architectures and scales. Note that in FedGraft, all the device models are grafted to a common rootstock, which enables knowledge aggregation on a tree-like global model. We use the notation “ X - Y ” to denote a reassembled device submodel whose rootstock and the remaining part come from X and Y , respectively. For example, we use M - S to denote a reassembled submodel where its rootstock comes from a Medium model, the remaining part comes from a Small model, and both parts are connected by our proposed adapter mechanism.

3) *Device Heterogeneity Settings*: To model device heterogeneity regarding memory constraints, we classified the 100 devices into three categories (i.e., *weak*, *medium*, and *strong*) based on their memory capacities. For the combination of models including CNN, MobileNetV2, and ResNet18, we assume that weak, medium, and strong devices can host models with no more than 1 M, 5 M, and 20 M parameters, respectively. For the combination of models including MobileNetV2, ResNet18, and VGG16, we assume that weak, medium, and strong devices can host models with no more than 5 M, 15 M, and 35 M parameters, respectively. By default, the ratio of devices in the three categories is set to 4:4:2.

4) *Baseline Method Settings*: We compared FedGraft with the following five baseline methods: i) *FedAvg* [2], the classic FL method that discards a dispatched model if the corresponding selected device cannot accommodate it; ii) *FedDF* [21], a KD-based FL method that uses 1% of the training data as the global proxy dataset on the server for knowledge distillation; iii) *HeteroFL* [22], a heterogeneous FL framework based on

TABLE II
PERFORMANCE COMPARISON OF DIFFERENT FL METHODS ON CIFAR-10

Method	Model	Accuracy (%)			Params (M)
		IID	$\beta = 0.6$	$\beta = 0.3$	
FedAvg	S	67.84±0.13	64.66±0.29	63.16±0.50	0.21
	M	58.28±0.18	60.11±0.52	48.21±1.23	2.25
	L	53.21±0.19	48.56±0.24	45.95±0.41	11.17
FedDF	S	67.68±0.15	66.73±0.18	63.34±0.42	0.21
	M	63.56±0.11	64.66±0.34	60.99±0.66	2.25
	L	58.60±0.22	59.06±0.37	49.83±0.93	11.17
HeteroFL	S_L	73.38±0.22	67.90±0.41	62.91±0.83	1.07
	M_L	69.72±0.17	63.47±0.25	57.15±0.62	2.90
	L_L	58.90±0.16	53.05±0.47	48.68±0.67	11.17
ScaleFL	S_L	74.97±0.13	69.53±0.60	64.84±1.99	1.01
	M_L	72.45±0.12	66.50±0.62	61.82±2.30	2.80
	L_L	62.32±0.16	57.61±0.70	52.62±0.75	11.18
FedBase	S-S	65.77±0.14	60.94±0.49	60.19±0.29	0.22
	S-M	62.01±0.10	59.07±0.53	49.49±1.15	2.20
	S-L	54.48±0.14	50.45±0.19	48.73±0.23	10.52
	M-S	68.36±0.16	65.19±0.27	61.13±0.33	0.28
	M-M	59.77±0.22	58.05±0.64	49.65±1.35	2.26
	M-L	48.83±0.20	46.62±0.23	43.58±0.18	10.58
	L-S	73.38±0.29	68.97±0.40	66.72±0.31	0.91
	L-M	68.39±0.21	64.77±0.45	57.72±1.28	2.89
	L-L	55.28±0.22	50.57±0.36	47.60±0.31	11.21
FedGraft	S-S	67.58±0.25	65.00±0.50	61.80±0.47	0.22
	S-M	63.89±0.21	60.06±0.42	52.45±2.53	2.20
	S-L	57.78±0.12	55.05±0.68	52.54±0.32	10.52
	M-S	67.75±0.21	65.37±0.28	60.06±0.53	0.28
	M-M	66.95±0.14	63.82±1.26	52.62±3.95	2.26
	M-L	64.22±0.10	61.72±0.46	55.93±0.56	10.58
	L-S	73.82±0.11	72.32±0.45	68.52±0.79	0.91
	L-M	75.62±0.10	72.34±0.82	65.55±3.14	2.89
	L-L	69.54±0.08	66.61±0.69	64.66±0.74	11.21

model width-wise pruning; iv) *ScaleFL* [36], a heterogeneous FL framework based on both model width- and depth-wise pruning; and v) *FedBase*, a FedAvg-like method based only on a specific kind of reassembled submodels extracted from the global model of FedGraft.

B. Performance Comparison (RQ1)

Table II compares the classification performance between FedGraft and the five FL baselines on the CIFAR-10 dataset under different data distributions (i.e., IID, $\beta=0.6$, and $\beta=0.3$), where we highlight the top results with underlines and indicate the best-performing models of similar sizes in bold. Note that unlike FedAvg and FedBase, which depend on a specific client model for global model training, the other four FL methods incorporate three distinct types of device models in the FL training process. Especially for FedGraft, we investigated its performance using three sets of heterogeneous models, each set having a common rootstock for the models. Moreover, the small and medium models in HeteroFL and ScaleFL differ from those in FedAvg and FedDF since they are obtained based on pruning rather than partitioning. To highlight these distinctions, we used different notations for models in the form of X_L ($X \in \{S, M, L\}$) to indicate that a sub-model of L with a size of X . From this table, we can find that when the rootstock of reassembled submodels (in the form of “ L - Y ”, $Y \in \{S, M, L\}$)

TABLE III
PERFORMANCE COMPARISON OF DIFFERENT FL METHODS ON DIFFERENT DATASETS AND DISTRIBUTIONS WITH L-BASED SUBMODELS

Method	Model	Accuracy (%) on CIFAR-10			Accuracy (%) on CIFAR-100			Accuracy (%) on TinyImageNet			Params (M)
		IID	$\beta = 0.6$	$\beta = 0.3$	IID	$\beta = 0.6$	$\beta = 0.3$	IID	$\beta = 0.6$	$\beta = 0.3$	
FedAvg	S	67.84±0.13	64.66±0.29	63.16±0.50	23.41±0.09	25.20±0.20	25.15±0.12	11.87±0.10	13.95±0.20	15.38±0.14	0.21
	M	58.28±0.18	60.11±0.52	48.21±1.23	21.81±0.21	24.68±0.14	23.36±0.20	23.23±0.27	23.67±0.22	22.72±0.25	2.25
	L	53.21±0.19	48.56±0.24	45.95±0.41	20.90±0.20	19.59±0.43	18.26±0.22	27.57±0.23	26.25±0.43	24.69±0.44	11.17
FedDF	S	67.68±0.15	66.73±0.18	63.34±0.42	26.94±0.13	29.56±0.17	28.84±0.28	19.69±0.22	20.73±0.27	21.47±0.24	0.21
	M	63.56±0.11	64.66±0.34	60.99±0.66	25.44±0.16	25.90±0.24	25.57±0.11	30.51±0.18	29.99±0.17	30.16±0.24	2.25
	L	58.60±0.22	59.06±0.37	49.83±0.93	22.44±0.23	21.92±0.26	21.08±0.53	26.24±0.30	24.18±0.25	22.29±0.40	11.17
HeteroFL	S _L	73.38±0.22	67.90±0.41	62.91±0.83	32.71±0.23	32.84±0.25	32.98±0.12	29.01±0.12	28.73±0.24	27.76±0.12	1.07
	M _L	69.72±0.17	63.47±0.25	57.15±0.62	29.73±0.20	30.03±0.35	28.88±0.28	28.94±0.25	28.90±0.32	27.36±0.44	2.90
	L _L	58.90±0.16	53.05±0.47	48.68±0.67	22.11±0.16	20.93±0.34	20.04±0.19	22.57±0.40	20.90±0.35	20.04±0.31	11.17
ScaleFL	S _L	74.97±0.13	69.53±0.60	64.84±1.99	31.44±0.23	31.39±0.20	32.36±0.08	26.15±0.06	26.53±0.15	24.85±0.32	1.01
	M _L	72.45±0.16	66.50±0.62	61.82±2.30	31.60±0.17	31.30±0.19	30.91±0.33	30.47±0.12	30.97±0.23	29.55±0.48	2.80
	L _L	62.32±0.16	57.61±0.70	52.62±0.75	23.34±0.29	21.50±0.47	20.02±0.20	27.39±0.36	24.82±0.39	21.68±0.74	11.18
FedBase	L-S	73.38±0.29	68.97±0.40	66.72±0.31	35.80±0.22	38.84±0.24	34.91±0.34	22.67±0.22	24.20±0.18	23.96±0.23	0.91
	L-M	68.39±0.21	64.77±0.45	57.72±1.28	24.18±0.17	24.80±0.28	23.73±0.24	19.70±0.23	21.81±0.28	20.77±0.22	2.89
	L-L	55.28±0.22	50.57±0.36	47.60±0.31	20.44±0.17	19.55±0.33	18.45±0.14	29.24±0.13	27.60±0.32	26.43±0.48	11.21
FedGraft	L-S	73.82±0.11	72.32±0.45	68.52±0.79	36.46±0.22	38.00±0.28	36.60±0.37	24.81±0.25	26.28±0.24	25.20±0.21	0.91
	L-M	75.62±0.10	72.34±0.82	65.55±3.14	31.44±0.30	31.82±0.31	31.72±0.23	30.69±0.26	31.01±0.29	29.17±0.32	2.89
	L-L	69.54±0.08	66.61±0.69	64.66±0.74	25.17±0.22	26.35±0.15	24.94±0.14	29.77±0.25	28.60±0.24	27.73±0.31	11.21

comes from the large model (i.e., ResNet 18), FedGraft outperforms other FL methods significantly. In other words, without imposing any extra assumptions on training data and model architecture, FedGraft can still beat those FL methods with such assumptions, reflecting the effectiveness of our approach within both IID and non-IID scenarios.

From the results of FedBase and FedGraft in Table II, we can also find that when the rootstock is extracted from a larger model, the overall classification performance for different device models gets better. This is mainly because a rootstock from a larger model indicates a feature-extraction block with better capability in extracting features. For example, when adopting the m^{ex} block of ResNet18 as the rootstock, FedGraft can achieve the best performance compared with those that use the m^{ex} blocks of CNN and MobileNetV2 as the rootstocks. In this case, although the submodels in the form of “L-Y” ($Y \in \{S, M, L\}$) are slightly larger than their counterparts in the forms of “S-Y” and “M-Y”, such an overhead is tolerable due to their significant improvement in classification accuracy. For example, although the reassembled model L-L has 0.69 M more parameters than S-L, it can achieve more than 11% accuracy improvement in FedGraft for all three data distributions. Note that in FedGraft, a smaller submodel (e.g., L-S) typically has better classification performance than its larger counterparts (e.g., L-L) since it can be dispatched to more devices for sufficient local training.

C. Adaptivity Analysis (RQ2)

1) *Adaptivity to Different Datasets and Data Distributions:* To evaluate the adaptivity of FedGraft, we conducted experiments on three datasets (i.e., CIFAR-10, CIFAR-100, and TinyImageNet), where each dataset involves one IID and two non-IID scenarios ($\beta = 0.6$ and $\beta = 0.3$). Table III shows the comparison results. Note that since Table II shows that reassembled submodels with the L-based rootstock can achieve the highest accuracy, here we only present the results using L-based submodels for FedBase and FedGraft. From this table, we can observe that FedGraft achieves the best performance over the other five FL methods in most cases. For TinyImageNet, the

table shows that the classification results of submodel L-S in FedGraft are slightly worse than those of the small model S_L in HeteroFL. This is mainly because the small pruning-based model S_L in HeteroFL has a larger size than L-S, which can lead to a better classification performance. To comprehensively compare the learning capabilities of different FL methods, Fig. 2 presents their learning curves with L-based submodels on the CIFAR-10 dataset. From the figure, we can clearly observe the superiority of FedGraft against other FL methods in both IID and non-IID scenarios.

2) *Adaptability to Different Device Models:* From Tables I and II, we can observe that the memory overhead caused by our proposed adapters is negligible. For example, the S model (i.e., CNN) in FedAvg requires 0.21 M parameters, while the S-S submodel in FedGraft needs 0.22 M parameters. In other words, the memory overhead posed by the adapter in the submodel S-S is 0.01 M parameters. Note that, as shown in Table I, the two building blocks of ResNet18 exhibit comparable FLOPs. To evaluate the adaptability of our approach to different combinations of models, we considered a new group of reassembled submodels, using MobileNetV2, ResNet18, and VGG16 [47] as the small, medium, and large candidate models, respectively. Table IV compares the performance between the five baselines and FedGraft on the CIFAR-10 dataset. We can find that although the two building blocks of VGG16 have significantly different FLOPs, FedGraft with L-based submodels can still achieve the best classification performance in most cases, which is consistent with the results shown in Tables II and III. Similarly, the results of M-S in FedGraft are better than those of L-S, since M-S is larger than L-S in this case. Note that, when adopting L-based submodels, the adapter implementation needs 0.03 M parameters.

D. Scalability Analysis (RQ3)

1) *Impact of Different Numbers of Devices:* To evaluate the scalability of FedGraft, Table V compares FedBase with FedGraft on the CIFAR-10 dataset using the L-based submodels, where the numbers of devices are set to 50, 100, 200, and

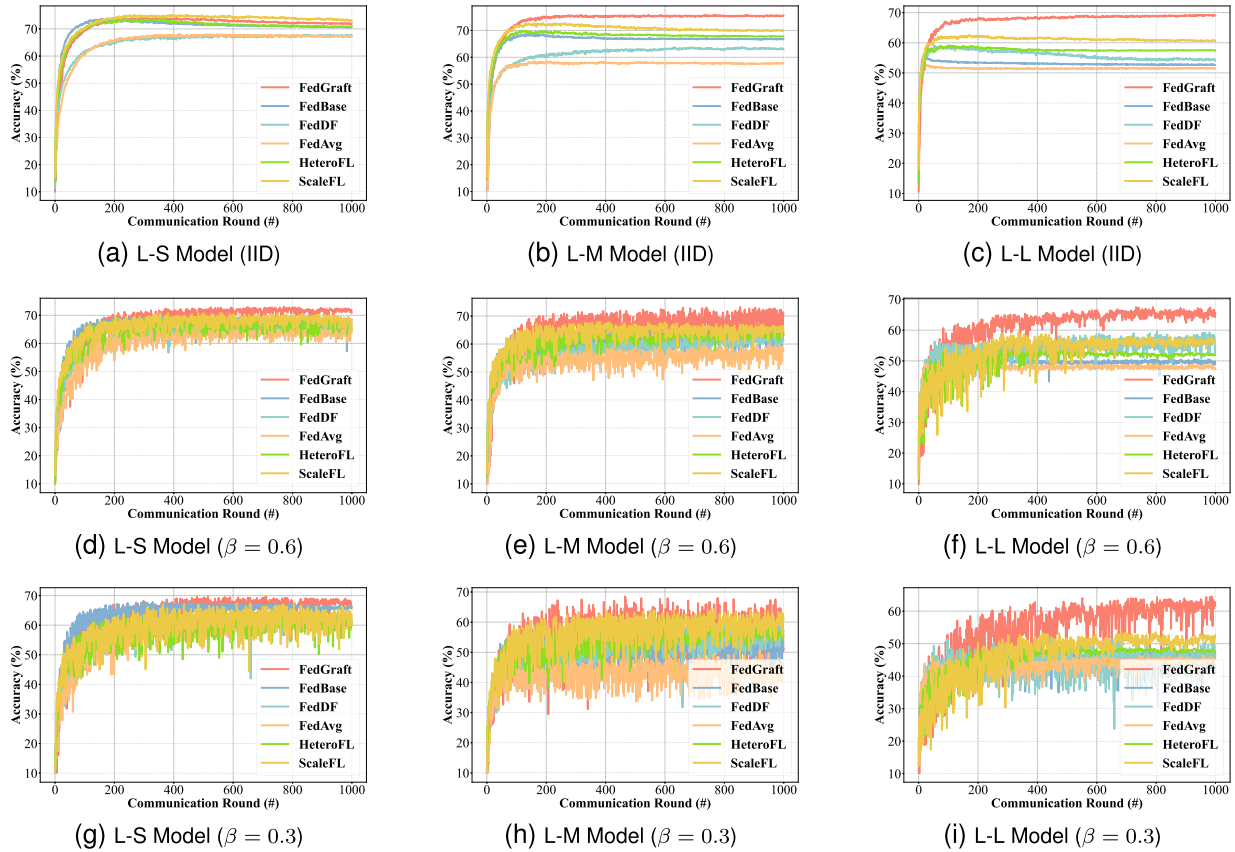


Fig. 2. Learning curves of different FL methods with L-based submodels on CIFAR-10.

500, respectively. From this table, we can find that FedGraft outperforms FedBase significantly in all cases due to the knowledge sharing among heterogeneous models enabled by FedGraft. Note that the FL training processes of FedBase rely on only a specific kind of device model, where the L-L submodel cannot be accommodated by medium and weak devices. Therefore, in FedBase, the L-L submodel performs worse than its L-M and L-S counterparts. As the number of devices increases, the overall classification performance of reassembled submodels decreases. This is because the total number of samples in the training dataset is fixed for each experiment, where more devices will lead to fewer data on each device, thus deteriorating classification performance.

2) *Impact of Different Ratios of Activated Devices:* Since the number of activated devices strongly affects the FL performance, we experimented to investigate the impact of different ratios of activated devices on FL training. Table VI compares the classification performance between FedBase and FedGraft on the dataset CIFAR-10 within an IID scenario, where the ratios (i.e., α) of activated devices are set to 0.05, 0.1, 0.5, and 1.0, respectively. From this table, we can see that FedGraft can always achieve the best results compared to FedBase. Since more heterogeneous models are involved in each FL communication round, as the ratio of activated devices increases, the overall classification performance of submodels in FedGraft will decrease slightly.

3) *Impacts of Different Proportions of Heterogeneous Devices:* Since the proportion of heterogeneous devices strongly affects the dispatch of reassembled submodels to devices, it plays an important role in determining the overall classification performance of submodels. Table VII presents the performance comparison results considering different proportions of heterogeneous devices using L-based submodels. From this table, we can find that when more strong devices are involved in heterogeneous FL, FedGraft can achieve a better overall classification performance. This is mainly because stronger devices can accommodate more types of submodels, leading to more sufficient local training for the submodels and knowledge sharing among heterogeneous models. Moreover, we can find that FedGraft can always obtain the best results in all cases, showing the scalability of FedGraft under different proportions of heterogeneous devices.

E. Applicability Validation (RQ4)

The following experiments investigate the performance of FedGraft on real-world datasets and platforms to evaluate its applicability in real-world settings.

1) *Real-World Dataset Widar:* Table VIII shows the experimental results on a real-world IoT dataset, Widar [48], a public WiFi-based gesture dataset. We can find that, compared with FedBase, FedGraft can achieve better results under different data

TABLE V
PERFORMANCE COMPARISON ON CIFAR-10 USING DIFFERENT SUBMODELS
(S: MOBILENETV2, M: RESNET18, L: VGG16)

Method	Model	Accuracy (%)			Params (M)
		IID	$\beta = 0.6$	$\beta = 0.3$	
FedAvg	S	64.61±0.10	63.01±1.10	53.98±1.67	2.25
	M	64.91±0.18	66.19±0.14	58.91±0.22	11.17
	L	66.40±0.08	63.56±0.17	59.71±0.16	33.65
FedDF	S	71.60±0.10	69.21±0.48	66.49±0.56	2.25
	M	69.64±0.09	67.72±0.29	65.19±0.35	11.17
	L	69.82±0.13	68.29±0.33	62.46±0.30	33.65
HeteroFL	S _L	68.68±0.12	62.67±0.59	59.20±0.94	2.11
	M _L	75.01±0.11	72.66±0.11	69.76±0.13	10.16
	L _L	70.61±0.07	66.44±0.06	62.97±0.12	33.65
ScaleFL	S _L	73.60±0.13	65.82±0.48	58.60±0.79	2.88
	M _L	75.21±0.12	68.08±0.69	61.50±1.24	11.45
	L _L	72.39±0.11	69.03±0.10	64.18±0.27	33.65
FedBase	S-S	66.09±0.22	62.39±0.58	54.33±1.44	2.26
	S-M	61.54±0.18	60.18±0.53	52.40±1.51	10.58
	S-L	56.67±0.17	53.05±0.27	48.22±0.30	33.46
	M-S	72.31±0.13	69.12±0.63	62.31±1.55	2.89
	M-M	65.47±0.20	67.91±0.11	59.70±0.35	11.21
	M-L	66.86±0.12	62.41±0.17	57.18±0.25	34.10
	L-S	71.86±0.06	69.69±0.56	60.65±1.33	2.48
	L-M	71.10±0.10	72.95±0.12	69.32±0.18	10.79
	L-L	68.26±0.13	65.17±0.29	60.45±0.22	33.68
FedGraft	S-S	67.14±0.17	64.24±1.95	56.48±2.33	2.26
	S-M	66.50±0.09	66.02±0.34	60.47±0.86	10.58
	S-L	63.70±0.07	59.09±0.44	53.68±0.47	33.46
	M-S	75.63±0.14	73.14±1.28	65.04±2.14	2.89
	M-M	69.85±0.13	71.60±0.36	65.00±0.38	11.21
	M-L	71.20±0.08	67.86±0.55	63.17±0.72	34.10
	L-S	74.67±0.05	72.90±0.69	63.12±3.61	2.48
	L-M	75.30±0.08	75.01±0.17	70.75±0.28	10.79
	L-L	73.07±0.06	71.19±0.40	64.54±0.36	33.68

TABLE V
PERFORMANCE COMPARISON ON CIFAR-10 (IID) CONSIDERING DIFFERENT
TOTAL NUMBERS OF DEVICES

Method	Model	Accuracy (%) considering # of Devices			
		50	100	200	500
FedBase	L-S	76.29±0.15	73.38±0.29	70.58±0.19	64.98±0.20
	L-M	70.98±0.19	68.39±0.21	64.50±0.23	58.24±0.13
	L-L	57.07±0.09	55.28±0.22	51.86±0.10	49.04±0.10
FedGraft	L-S	77.72±0.10	73.82±0.11	70.79±0.08	66.73±0.12
	L-M	79.09±0.10	75.62±0.10	71.86±0.18	65.04±0.13
	L-L	73.97±0.09	69.54±0.08	64.26±0.07	58.89±0.10

distributions. Notably, the large models (i.e., L-L) in FedBase suffer from low classification accuracy since they cannot learn from weak and medium devices. However, using FedGraft, the large models can improve the classification accuracy by up to 10.39%.

2) *Real Test-Bed Platform*: All the above results were obtained from simulation experiments conducted on the workstation equipped with an Intel i9 CPU and an NVIDIA RTX 3090Ti GPU. To show the performance of our approach on real-world MEC applications, we experimented on a test-bed platform, which consists of 8 Jetson Nano boards and 2 Jetson Xavier AGX boards as devices and a workstation with an NVIDIA RTX 3080 GPU as the server. Table IX presents the detailed settings for the platform. Since only ten real devices are involved in

TABLE VI
PERFORMANCE COMPARISON ON CIFAR-10 (IID) CONSIDERING DIFFERENT
RATIOS OF ACTIVATED DEVICES

Method	Model	Accuracy (%)			
		0.05	0.1	0.5	1
FedBase	L-S	73.49±0.19	73.38±0.29	72.55±0.07	72.17±0.07
	L-M	69.47±0.14	68.39±0.21	67.45±0.10	67.23±0.15
	L-L	55.84±0.20	55.28±0.22	53.85±0.23	54.76±0.20
FedGraft	L-S	75.11±0.15	73.82±0.11	73.62±0.08	73.14±0.09
	L-M	76.46±0.12	75.62±0.10	74.09±0.16	73.09±0.13
	L-L	73.01±0.06	69.54±0.08	66.42±0.06	65.35±0.02

TABLE VII
PERFORMANCE COMPARISON ON CIFAR-10 (IID) WITH DIFFERENT
PROPORTIONS OF HETEROGENEOUS DEVICES (WEAK/MEDIUM/STRONG)

Method	Model	Accuracy (%)				
		0.8/0.1/0.1	0.4/0.4/0.2	0.3/0.3/0.4	0.2/0.4/0.4	0.1/0.1/0.8
FedBase	L-S	73.17±0.21	68.97±0.40	73.34±0.21	73.11±0.18	73.02±0.10
	L-M	56.96±0.18	64.77±0.45	69.61±0.17	70.49±0.11	72.03±0.16
	L-L	48.26±0.20	50.57±0.36	61.76±0.16	62.80±0.18	69.69±0.17
FedGraft	L-S	73.24±0.19	72.32±0.45	74.33±0.11	75.77±0.18	76.83±0.06
	L-M	72.83±0.20	72.34±0.82	77.06±0.12	75.74±0.13	79.32±0.09
	L-L	68.46±0.14	66.61±0.69	72.35±0.05	70.72±0.08	72.16±0.07

TABLE VIII
PERFORMANCE COMPARISON ON THE WIDAR DATASET

Method	Model	Accuracy (%)			Params (M)
		IID	$\beta = 0.6$	$\beta = 0.3$	
FedBase	L-S	57.53±0.32	53.69±0.82	51.86±0.59	0.91
	L-M	60.95±0.19	59.24±0.24	53.27±0.84	2.89
	L-L	52.05±0.33	51.13±0.25	49.90±0.23	11.21
FedGraft	L-S	59.49±0.21	56.36±0.47	53.06±1.08	0.91
	L-M	63.09±0.29	60.59±0.82	55.85±0.96	2.89
	L-L	62.44±0.22	59.67±0.40	58.15±0.44	11.21

TABLE IX
REAL TEST-BED PLATFORM SETTINGS

Type	Device	Compute Unit	Number
Device-Weak	Jetson Nano (w/ CPU)	ARM Cortex-A57 CPU	4
Device-Medium	Jetson Nano (w/ GPU)	128-core Maxwell GPU	4
Device-Strong	Jetson Xavier AGX	512-core NVIDIA GPU	2
Server	Workstation	NVIDIA RTX 3080 GPU	1

FL, in each communication round, we assumed that 50% of the devices participate in the FL training.

Fig. 3(a) presents the edge devices used in the test-bed platform, where four Nano boards (with CPU) were used as weak devices, and the remaining four Nano boards (with GPU) were used as medium devices. We used Xavier AGX boards as strong devices. Fig. 3(b)–(d) shows the training curves of different submodels on the CIFAR-10 dataset within an IID scenario. From these subfigures, we can find that FedGraft performs better than its FedBase counterparts with different underlying training models, showing the superiority of FedGraft in dealing with collaborative learning between devices with heterogeneous models.

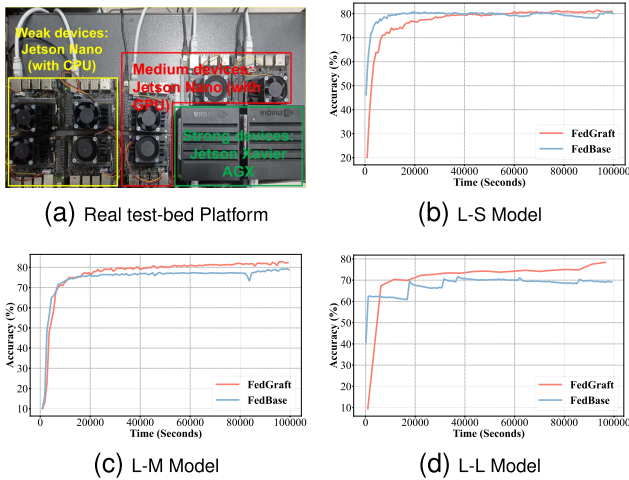


Fig. 3. Learning curves of models on CIFAR-10 on the real test-bed platform.

VI. CONCLUSION

Along with the prosperity of Artificial Intelligence (AI) and the awareness of data privacy, federated learning is becoming increasingly popular in the design of AIoT applications. However, due to the lack of efficient collaborative training schemes for heterogeneous deep models hosted by AIoT devices, existing FL methods suffer from low inference performance, especially when device memory sizes vary. As a result, the large-scale development of FL-based AIoT applications is greatly limited. To solve this problem, this paper presented a novel heterogeneous FL method named FedGraft based on our proposed model partitioning and grafting mechanism. Based on a tree-like global model and our proposed device selection strategy for grafted models, the overall FL inference performance can be improved without imposing unrealistic assumptions. Comprehensive experiments demonstrated the adaptability, scalability, and applicability of our FedGraft approach on both simulated and real test-bed platforms. Since device data distributions play an important role in determining FL training performance, in the future we plan to investigate effective methods to further explore FedGraft's performance when dealing with various non-IID scenarios.

ACKNOWLEDGMENT

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore, Cyber Security Agency of Singapore as well as CyberSG R&D Programme Office, Singapore.

REFERENCES

- [1] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist. Conf.*, 2017, pp. 1273–1282.
- [3] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1/2, pp. 1–210, 2021.
- [4] Z. Wang, Q. Hu, R. Li, M. Xu, and Z. Xiong, "Incentive mechanism design for joint resource allocation in blockchain-based federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1536–1547, May 2023.
- [5] M. Hu et al., "FedMut: Generalized federated learning via stochastic mutation," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 12528–12537.
- [6] X. Zhang, M. Hu, J. Xia, T. Wei, M. Chen, and S. Hu, "Efficient federated learning for cloud-based AIoT applications," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 40, no. 11, pp. 2211–2223, Nov. 2021.
- [7] T. Zeng, O. Semiari, M. Chen, W. Saad, and M. Bennis, "Federated learning on the road autonomous controller design for connected and autonomous vehicles," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 10407–10423, Dec. 2022.
- [8] A. Nguyen et al., "Deep federated learning for autonomous driving," in *Proc. IEEE Intell. Veh. Symp.*, 2022, pp. 1824–1830.
- [9] L. Li, H. Xiong, Z. Guo, J. Wang, and C.-Z. Xu, "SmartPC: Hierarchical pace control in real-time federated learning system," in *Proc. IEEE Real-Time Syst. Symp.*, 2019, pp. 406–418.
- [10] M. Hu et al., "GitFL: Uncertainty-aware real-time asynchronous federated learning using version control," in *Proc. IEEE Real-Time Syst. Symp.*, 2023, pp. 145–157.
- [11] M. Duan, D. Liu, X. Chen, R. Liu, Y. Tan, and L. Liang, "Self-balancing federated learning with global imbalanced data in mobile systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 59–71, Jan. 2021.
- [12] M. Hu, E. Cao, H. Huang, M. Zhang, X. Chen, and M. Chen, "AIoTML: A unified modeling language for AIoT-based cyber-physical systems," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 42, no. 11, pp. 3545–3558, Nov. 2023.
- [13] Y. Zeng, Y. Lin, Y. Yang, and J. Liu, "Differentially private federated temporal difference learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2714–2726, Nov. 2022.
- [14] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "A survey on federated learning for resource-constrained IoT devices," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 1–24, Jan. 2022.
- [15] C. Wang, Y. Yang, and P. Zhou, "Towards efficient scheduling of federated mobile devices under computational and statistical heterogeneity," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 394–410, Feb. 2021.
- [16] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.
- [17] M. Hu, W. Duan, M. Zhang, T. Wei, and M. Chen, "Quantitative timing analysis for cyber-physical systems using uncertainty-aware scenario-based specifications," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 11, pp. 4006–4017, Nov. 2020.
- [18] J. Wang, J. Hu, J. Mills, G. Min, M. Xia, and N. Georgalas, "Federated ensemble model-based reinforcement learning in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 6, pp. 1848–1859, Jun. 2023.
- [19] S. Wang et al., "Adaptive federated learning in resource constrained edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1205–1221, Jun. 2019.
- [20] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," 2019, *arXiv: 1910.03581*.
- [21] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 2351–2363.
- [22] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=TNkPBBYfKXg>
- [23] M. Kim, S. Yu, S. Kim, and S.-M. Moon, "DepthFL: Depthwise federated learning for heterogeneous clients," in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=pf8RIZTMU58>
- [24] J. Hong, H. Wang, Z. Wang, and J. Zhou, "Efficient split-mix federated learning for on-demand and in-situ customization," in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=_QLmakITKq
- [25] J. Xia, Y. Zhang, Z. Yue, M. Hu, X. Wei, and M. Chen, "HierarchyFL: Heterogeneous federated learning via hierarchical self-distillation," 2022, *arXiv:2212.02006*.
- [26] N. Passalis and A. Tefas, "Learning deep representations with probabilistic knowledge transfer," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 283–299.

- [27] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 3320–3328.
- [28] M. Long, Y. Cao, J. Wang, and M. Jordan, "Learning transferable features with deep adaptation networks," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 97–105.
- [29] K. Wang et al., "FlexiFed: Personalized federated learning for edge clients with heterogeneous model architectures," in *Proc. ACM Web Conf.*, 2023, pp. 2979–2990.
- [30] X. Yang, D. Zhou, S. Liu, J. Ye, and X. Wang, "Deep model reassembly," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 25739–25753.
- [31] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning (still) requires rethinking generalization," *Commun. ACM*, vol. 64, no. 3, pp. 107–115, 2021.
- [32] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
- [33] J. Xia, T. Liu, Z. Ling, T. Wang, X. Fu, and M. Chen, "PervasiveFL: Pervasive federated learning for heterogeneous IoT systems," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4100–4111, Nov. 2022.
- [34] S. Kang, W. Kwon, D. Lee, J. Lian, X. Xie, and H. Yu, "Distillation from heterogeneous models for top-K recommendation," in *Proc. ACM Web Conf.*, 2023, pp. 801–811.
- [35] Y. Jiang et al., "Model pruning enables efficient federated learning on edge devices," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 10374–10386, Dec. 2023.
- [36] F. Ilhan and G. Su, and L. Liu, "ScaleFL: Resource-adaptive federated learning with heterogeneous clients," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, CVPR, 2023, pp. 24532–24541.
- [37] Z. Jiang et al., "Computation and communication efficient federated learning with adaptive model pruning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2003–2021, Mar. 2024.
- [38] R. Liu et al., "No one left behind: Inclusive federated learning over heterogeneous devices," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 3398–3406.
- [39] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3519–3529.
- [40] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *Proc. Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=HJxNANVtDS>
- [41] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, "Sparsified SGD with memory," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4452–4463.
- [42] H. Yu, S. Yang, and S. Zhu, "Parallel restarted SGD with faster convergence and less communication: Demystifying why model averaging works for deep learning," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 5693–5700.
- [43] S. Rudner, "An overview of gradient descent optimization algorithms," 2016, *arXiv:1609.04747*.
- [44] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv:1909.06335*.
- [45] M. Sandler, A. G. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.
- [46] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [47] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2015. [Online]. Available: <http://arxiv.org/abs/1409.1556>
- [48] S. Alam et al., "FedAIoT: A federated learning benchmark for artificial intelligence of things," 2023, *arXiv:2310.00109*.



Ruixuan Liu received the BE degree from the Department of Software Engineering, School of Computer Science and Technology, Xidian University, Xian, China, in 2022. He is currently working toward the ME degree with the Software Engineering Institute, East China Normal University, Shanghai, China. His research interests are in the areas of AIoT applications, edge computing, and federated learning.



Ming Hu (Member, IEEE) received the BE and PhD degrees from the Software Engineering Institute, East China Normal University, Shanghai, China, in 2017 and 2022, respectively. He is currently a research scientist with Singapore Management University (SMU). Previously, he was a research fellow with Nanyang Technological University (NTU), Singapore. His research interests include the area of design automation of cyber-physical systems, federated learning, trustworthy AI, and software testing.



Zeke Xia received the BE degree from the Department of Software Engineering, School of Computer Science and Technology, Soochow University, Suzhou, China, in 2022. He is currently working toward the ME degree with the Software Engineering Institute, East China Normal University, Shanghai, China. His research interests include federated learning and AIoT applications.



Xiaofei Xie (Member, IEEE) received the PhD degree from Tianjin University and won the CCF Outstanding Doctoral Dissertation Award in China, in 2019. He is an assistant professor with Singapore Management University. Previously, he was a Wallenberg-NTU presidential postdoctoral fellow with NTU. His research mainly focuses on the quality assurance of both traditional software and AI-enabled software. He has published some top-tier conference/journal papers in the areas of software engineering, security and AI, such as ICSE, ESEC/FSE, NeurIPS, ICML, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *Usenix Security* and CCS. In particular, he has received four ACM SIGSOFT Distinguished Paper Awards (FSE'16, ASE'19, ISSTA'22 and ASE'23).



Jun Xia (Member, IEEE) received the BS degree in computer science and technology from Hainan University, Haikou, China, in 2016, the MS degree in computer science and technology from Jiangnan University, Wuxi, China, in 2019, and the PhD degree from Software Engineering Institute, East China Normal University, Shanghai, China, in 2023. He is currently a postdoc with the Department of Computer Science and Engineering, University of Notre Dame, USA. His research interests are in the areas of embedded systems, AIoT applications, and trustworthy computing.



Pengyu Zhang received the BE degree from the Chinese University of Mining & Technology-Beijing, Beijing, China, in 2019, and the MS degree from the University of Florida, Gainesville, FL, USA, in 2021. He is currently working toward the PhD degree with the Software Engineering Institute, East China Normal University, Shanghai, China. His areas of research interest include federated learning, deep learning model compression, and efficient model inference.



Yihao Huang (Member, IEEE) received the BS and PhD degrees from the Software Engineering Institute, East China Normal University, China, in 2017 and 2022, respectively. He is a research fellow with National University of Singapore (NUS), working on AI security. Previously, he was a research fellow with Nanyang Technological University (NTU), Singapore. He received the Best Paper Award in the ECCV 2022 AROW workshop. His research interests include computer vision and AI security, especially adversarial attacks and generative AI.



Mingsong Chen (Senior Member, IEEE) received the BS and ME degrees from the Department of Computer Science and Technology, Nanjing University, Nanjing, China, in 2003 and 2006, respectively, and the PhD degree in computer engineering from the University of Florida, Gainesville, in 2010. His research interests include the design automation of intelligent cyber-physical systems, trustworthy AI, hardware/software co-design, trustworthy AI, multi-modal embedded computing, and formal verification techniques. He is currently a professor with the Software Engineering Institute, East China Normal University and serves as the director with the Engineering Research Center of Software/Hardware Co-design Technology and Application affiliated with the Ministry of Education, China, and the vice director of the technical committee of embedded systems of China Computer Federation (CCF). He is an associate editor of *IEEE Transactions on Computers*.